

## **Comprehensive Methods for the Analysis of Biomedical Image (Plant Leaf)**

<sup>1</sup>Sumanta Karmakar, <sup>2\*</sup>Gurjeet Singh, <sup>3</sup>Anish Deb, <sup>4</sup>Ayushi Ankita Rakshit,  
<sup>5</sup>Dibyoyoti Mukherjee

<sup>1,2,3</sup>Assistant Professor, <sup>4,5</sup>Student

<sup>1245</sup>Department of Electronics & Communication Engineering

<sup>3</sup>Department of Mechanical Engineering  
Asansol Engineering College

**\*Corresponding author**

**E-mail Id:- gurjeetsingh714@gmail.com**

### **ABSTRACT**

Agriculture serves as the backbone of India's economy, providing employment opportunities for a significant portion of the population. However, the sector faces challenges such as manual cultivation practices and limited technical knowledge among farmers, leading to suboptimal crop yields. Additionally, the prevalence of heterogeneous diseases affecting plant leaves contributes to reduced agricultural productivity and economic losses. Current approaches relying on indiscriminate pesticide use pose risks to human health and the environment. Therefore, there is an urgent need to adopt advanced techniques for the early detection of plant diseases to ensure sustainable agricultural practices. This paper presents a comprehensive survey of various plant diseases affecting crops in India and explores advanced techniques for their detection, emphasizing the importance of timely intervention to mitigate losses and improve crop yields. The adoption of innovative disease detection methods holds the potential to revolutionize agricultural practices, promoting both economic prosperity and environmental sustainability.

**Keywords:-**Crops, Cultivation, pesticides, Plant diseases.

### **INTRODUCTION**

Biomedical imaging plays a crucial role in modern healthcare, enabling clinicians and researchers to visualize and understand the intricate structures and functions of the human body at various scales. From microscopic examination of cellular structures to macroscopic evaluation of organs and tissues, biomedical imaging techniques provide invaluable information for disease diagnosis, treatment planning, and medical research. However, the sheer volume and complexity of biomedical images pose significant challenges for manual interpretation and analysis. In response, comprehensive methods for

the analysis of biomedical images have emerged, leveraging advanced computational techniques from fields such as computer vision, machine learning, and image processing.

These methods offer automated and objective approaches to extract quantitative information, detect abnormalities, and characterize biological phenomena from biomedical images. By harnessing the power of computational algorithms, researchers and clinicians can uncover subtle patterns, biomarkers, and associations that may not be readily apparent to the

human eye. Moreover, comprehensive image analysis techniques enable the integration of multi-modal imaging data, allowing for a more holistic understanding of biological processes and disease mechanisms.

In this paper, we provide an overview of the state-of-the-art methods for the analysis of biomedical images. We discuss various computational approaches, including image segmentation, feature extraction, classification, registration, and visualization, and their applications across different imaging modalities, such as magnetic resonance imaging (MRI), computed tomography (CT), microscopy, and molecular imaging. Furthermore, we examine the challenges and opportunities in the field, including the need for robust algorithms, standardized evaluation metrics, large-scale annotated datasets, and ethical considerations.

Through comprehensive methods for the analysis of biomedical images, we aim to advance our understanding of health and disease, improve clinical decision-making, and ultimately enhance patient care and outcomes. By bridging the gap between imaging technology and computational analysis, we can unlock new insights into complex biological systems and pave the way for personalized medicine and precision healthcare.

## **METHODOLOGY OF PROPOSED SYSTEM:**

This proposed model aims to detect defects in plants at the ground level, primarily through leaf analysis, to minimize damage and enhance agricultural yield. The process involves collecting a dataset of leaf images to identify diseases, followed by training a model for classification. Subsequently, the input images undergo pre-processing

before being classified, enabling the prediction of the disease type and recommendation of appropriate treatments. By leveraging machine learning and image analysis techniques, this system facilitates early disease detection and intervention, ultimately contributing to improved agricultural productivity and sustainability. We have designed our model in several phases as follows:

### **Dataset Collection and Model Training:**

- Collection of leaf images representing various diseases and healthy states
- Training a classification model using machine learning algorithms such as convolutional neural networks (CNN)
- Evaluation and validation of the trained model to ensure accuracy

### **Preprocessing and Classification:**

- Pre-processing techniques to enhance the quality of input images
- Classification of input images into disease categories using the trained model
- Integration of decision-making algorithms to predict disease types based on image analysis.

### **Treatment Recommendation:**

- Utilization of a database containing information on disease treatments and management practices.
- Integration of decision-support algorithms to recommend appropriate treatments based on the identified disease type.
- Consideration of factors such as crop type, environmental conditions, and cultural practices in treatment recommendations.

### **Implementation and Validation:**

- Deployment of the proposed system in agricultural settings for real-time disease detection.

- Validation of the system's performance through field trials and comparative studies.
- Assessment of the system's impact on agricultural productivity and economic outcomes.

**Future Directions and Challenges:**

- Opportunities for further refinement and optimization of the model.
- Challenges related to data collection, model scalability, and integration with existing agricultural practices.
- Potential expansion of the system to address additional agricultural challenges beyond disease detection.

The above steps depict the training and deployment phases of our leaf disease detection model. In Summary of the proposed model's capabilities in early disease detection and treatment recommendation. It defines its Implication for improving agricultural productivity, sustainability, and economic outcomes. The importance of continued research and development to advance the field of agricultural technology. By implementing this model, farmers can detect plant diseases early, apply timely treatments, and minimize crop losses, thereby enhancing overall agricultural productivity and contributing to food security and economic development.

The dataset is loaded first in the training phase which is done from Kaggle website. Training and modeling are streamlined during the training phase. So training is done using TensorFlow and Keras. The training dataset images are converted to their respective machine language format using CNN (Convolutional Neural Networks). CNN is a deep learning algorithm specially designed for working with images and videos. As CNN consists of three layers,

i.e., Input, Hidden and Output layer. The basic operation by which a machine understands whether a leaf is defected or healthy is done in Hidden layer part of Convolutional Neural Networks. Basically, there are 4 steps used to differentiate one image from the other. Firstly, Line up the features and the image. Secondly, Multiply each image pixel by corresponding feature pixel. Thirdly, Add the value and find the sum and lastly, divide the sum by the total number of pixels in the feature.

After processing dataset images to the disk, model is loaded to detect the disease of a leaf on the images. The trained model will be loaded in .h5 format which stands for Hierarchical Data Format (HDF). It contains multidimensional arrays of scientific data. Here, scientific data is referred as the values of the trained dataset images. Once the file is loaded in a disk, our model is ready to work. Now, the testing part is done with the help of OpenCV (CV2). To test an image, we have pasted the path of the image which we want to check. Comparison between the trained and the tested image is done and finally the result is predicted.

**MODULE DESCRIPTION:*****Machine Learning:***

Machine learning is a subset of artificial intelligence (AI) that focuses on developing algorithms and models that enable computers to learn from data and make predictions or decisions without being explicitly programmed. In other words, machine learning algorithms allow computers to identify patterns in data and learn from these patterns to perform tasks or make predictions.

***Data in Machine learning:***

Data is the fuel that powers machine learning algorithms. In the context of machine learning, data refers to the

information used to train, validate, and test models. It serves as the foundation upon which algorithms learn patterns, make predictions, and derive insights.

**Example:** Why did Facebook acquire WhatsApp by paying a huge price of \$19 billion? The answer is very simple and logical – it is to have access to the users' information that Facebook may not have but WhatsApp will have. This information of their users is of paramount importance to Facebook as it will facilitate the task of improvement in their services.

### ***Deep Learning :***

The term "deep" refers to a subtype of machine learning that extracts and learns features from data using multiple-layered artificial neural networks. These neural networks are composed of interconnected nodes (neurons) organized into layers, with each layer responsible for performing specific transformations on the input data. Deep learning has gained significant popularity and has demonstrated remarkable success in various domains, including computer vision, natural language processing, speech recognition, and reinforcement learning.

### **Example of Deep Learning at Work**

Assume that training a neural network to identify images with dogs is the objective. Dogs vary in appearance; for example, a Rottweiler and a Poodle don't look exactly same. In addition, photographs depict dogs from various perspectives and with differing levels of light and shadow. As a result, a training set of photos needs to be assembled, comprising numerous instances of dog faces that are universally recognized as "dogs," as well as images of non-dog items that are appropriately labeled "not dog." The Data is created by the neural network

using photos as input. As this data flow through the network, different nodes give distinct elements different weights. The seemingly unrelated information, such as "furry," "has a snout," "has four legs," etc., is combined in the final output layer to produce the output "dog."

The neural network's response will now be compared against the label that was created by humans. The output is verified if there is a match. The neural network records the error and modifies the weightings if it doesn't. The neural network continuously modifies its weights in an attempt to enhance its dog-recognition abilities. This method of training, known as supervised learning, takes place even in the absence of explicit information on what "makes" a dog for neural networks. They have to learn on their own and identify trends in data over time.

After defining deep learning and comprehending the fundamentals of how it functions, let's take a brief look back at its development.

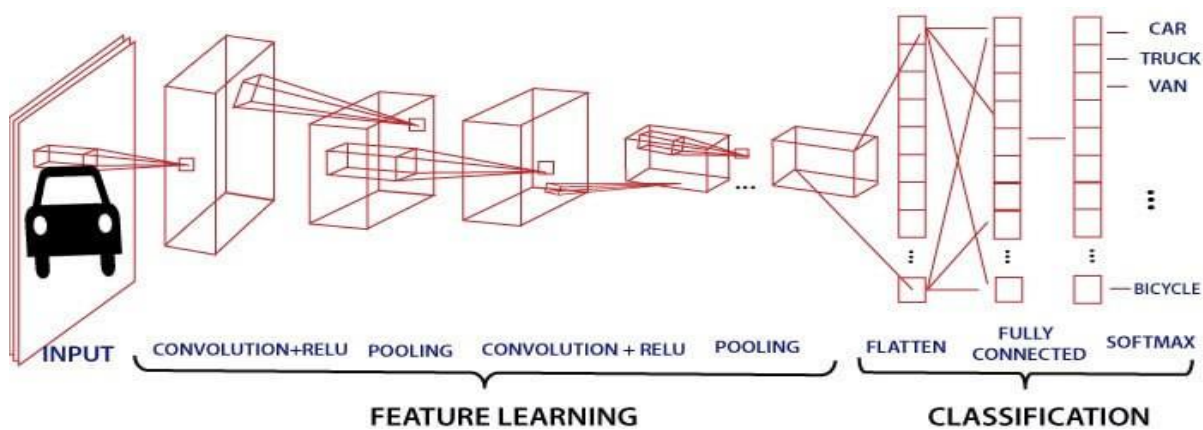
### ***CNN (Convolutional Neural Network):***

A subclass of deep neural networks called convolutional neural networks (CNNs) is especially made for handling structured, grid-like data, like photographs. CNNs have revolutionized the field of computer vision and are widely used for tasks such as image classification, object detection, facial recognition, and image segmentation. They leverage the concept of convolution to automatically learn hierarchical representations of features directly from pixel values.

Each input image in CNN is processed through a series of convolution layers, pooling, fully connected layers, and filters (sometimes referred to as kernels).

Next, we will categorize an item having probabilistic values of 0 and 1 by using the Soft-max function. OpenCV (CV2):

The following figure (fig 1) shows the structure for Convolution Neural Network.



**Fig 1: Convolution Neural Network**

### **OpenCV::**

OpenCV (Open Source Computer Vision Library) is an open-source library of programming functions mainly aimed at real-time computer vision tasks. Originally developed by Intel in 1999, it was later maintained by Willow Garage and is currently managed by Itseez, which was later acquired by Intel. OpenCV is written in C++ and includes interfaces for C, Python, Java, and MATLAB/Octave, making it accessible to a wide range of developers and researchers.

### **Grayscale**

Grayscale images are those images which contain only two colors black and white. The contrast measurement of intensity is black treated as the weakest intensity, and white as the strongest intensity. When we use the grayscale image, the computer assigns each pixel value based on its level of darkness.

### **RGB**

An RGB is a combination of the red, green, blue color which together makes a new color. The computer retrieves that value from each pixel and puts the results in an array to be interpreted. OpenCV is available for free of cost.

- Since the OpenCV library is written in C/C++, so it is quit fast. Now it can be used with Python.
- It require less RAM to usage, it maybe of 60-70 MB.
- Computer Vision is portable as OpenCV and can run on any device that can run on C.

### **Tensorflow:**

TensorFlow is an open-source machine learning framework developed by Google Brain for building and training various deep learning models. It provides a flexible and scalable platform for implementing a wide range of machine learning algorithms, from simple linear models to complex deep neural networks. TensorFlow is designed to be efficient, scalable, and suitable for both research and production-level deployments.

Key features and components of TensorFlow include:

### **Computational Graphs:**

TensorFlow represents computations as directed graphs, where nodes represent operations and edges represent data flow between operations. This graph-based approach allows for efficient execution



and optimization of complex computational tasks.

#### **Tensor Data Structure:**

TensorFlow operates on tensors, which are multi-dimensional arrays similar to NumPy arrays. Tensors can represent data of various types and shapes, such as scalars, vectors, matrices, or higher-dimensional arrays.

#### **Automatic Differentiation:**

TensorFlow provides automatic differentiation capabilities through its computational graph framework. This allows users to compute gradients of loss functions with respect to model parameters, enabling efficient optimization using gradient-based optimization algorithms like stochastic gradient descent (SGD).

#### **Flexible Deployment Options:**

TensorFlow supports deployment across a wide range of platforms and devices, including CPUs, GPUs, TPUs (Tensor Processing Units), desktops, servers, mobile devices, and cloud environments. It offers APIs for programming languages such as Python, C++, and JavaScript, making it accessible to developers across different platforms.

#### **High-Level APIs:**

Deep learning model development and training are made easier by TensorFlow's high-level APIs, which include TensorFlow Estimators and Keras, which are integrated within the framework. These APIs abstract away low-level implementation details and provide easy-to-use interfaces for common machine learning tasks.

#### **Model Serving and Deployment:**

TensorFlow Serving is a dedicated framework for deploying trained TensorFlow models in production environments. It provides efficient model

serving capabilities, including support for model versioning, load balancing, and monitoring.

#### **Community and Ecosystem:**

A sizable and vibrant community of academics, developers, and contributors actively participates in TensorFlow's ecosystem development. It offers extensive documentation, tutorials, and resources for learning and using TensorFlow effectively.

#### **Integration with Other Libraries and Frameworks:**

TensorFlow integrates seamlessly with other popular machine learning libraries and frameworks, such as scikit-learn, PyTorch, and Keras. This interoperability allows users to leverage the strengths of different tools and frameworks within their machine learning workflows. Overall, TensorFlow is a powerful and versatile framework for building and deploying machine learning models at scale. It has been widely adopted across academia and industry for a variety of applications, including image classification, natural language processing, reinforcement learning, and more. With its flexible architecture, extensive features, and strong community support, TensorFlow continues to be at the forefront of machine learning innovation.

#### **Keras:**

Keras is an open-source deep learning library written in Python that provides a high-level interface for building and training neural networks. Its development prioritized keeping user-friendliness and extensibility while facilitating quick experimentation and prototyping of deep learning models. Keras is designed to be user-friendly, modular, and easy to use, making it suitable for both beginners and experienced deep learning practitioners. Key features and characteristics of Keras

include:

**User-Friendly API:**

Keras offers a simple and intuitive API that allows users to quickly define and train deep learning models with just a few lines of code. It follows a consistent and easy-to-understand syntax, making it accessible to users with varying levels of expertise.

**Modularity and Extensibility:**

Keras adopts a modular design, with layers, models, optimizers, and other components implemented as modular building blocks. Users can easily assemble these components to create complex neural network architectures or extend Keras with custom layers and functionalities.

**Support for Multiple Backends:**

Keras provides support for multiple deep learning backend engines, including TensorFlow, Theano, and Microsoft Cognitive Toolkit (CNTK). This allows users to seamlessly switch between backend engines without modifying their code, depending on their specific requirements and preferences.

**Wide Range of Applications:**

Keras supports a wide range of deep learning applications, including image classification, object detection, natural language processing, sequence modeling, reinforcement learning, and more. It includes built-in implementations of popular neural network architectures, such as convolutional neural networks (CNNs), recurrent neural networks (RNNs), and deep belief networks (DBNs).

**Integration with TensorFlow:**

In TensorFlow 2.0 and later versions, Keras has become the official high-level API for building and training deep learning models. It is seamlessly

integrated with TensorFlow, allowing users to leverage the strengths of both frameworks for their machine learning projects.

**Community and Documentation:**

Keras has a large and active community of developers, researchers, and contributors who contribute to its development and ecosystem. It offers extensive documentation, tutorials, and examples to help users get started and learn how to use Keras effectively.

**Model Compatibility and Transfer Learning:**

Keras models are compatible with TensorFlow, enabling easy integration with TensorFlow-specific features and tools. Additionally, Keras supports transfer learning, allowing users to leverage pre-trained models and fine-tune them for specific tasks or domains with minimal effort.

Overall, Keras provides a user-friendly and versatile platform for building and training deep learning models. Its simplicity, flexibility, and integration with TensorFlow make it a popular choice for researchers, developers, and practitioners working on various machine learning projects and applications.

**Theano**

The MILA group created theano at the University of Montreal in Quebec, Canada. It is a popular open-source Python toolkit that combines scipy and numpy to enable the execution of mathematical operations on multi-dimensional arrays. By automatically creating symbolic graphs, it computes gradients more quickly by making use of GPUs. Since it computes unstable expressions using more stable algorithms after first observing them numerically, it has proven to be highly appropriate for such expressions.

**CNTK**

The open-source framework for deep learning is called Microsoft Cognitive Toolkit. It is made up of every fundamental component needed to create a neural network. Python or C++ are used to train the models, whereas C# or Java is used to load the model and make predictions.

**Numpy:**

Large, multi-dimensional arrays and matrices are supported by Numpy, a library for the Python programming language that also offers a sizable number of high-level mathematical functions that can be used to manipulate huge arrays.

Numpy is a non-optimizing bytecode interpreter. Mathematical algorithms written for this version of Python often run much slower than compiled equivalents. One way that NumPy tackles the slowness issue is by offering multidimensional arrays as well as functions and operators that work well with arrays; however, in order to use these, certain code, primarily inner loops, must be rewritten using NumPy.

**Matplotlib:**

For the Python programming language and its NumPy numerical mathematics

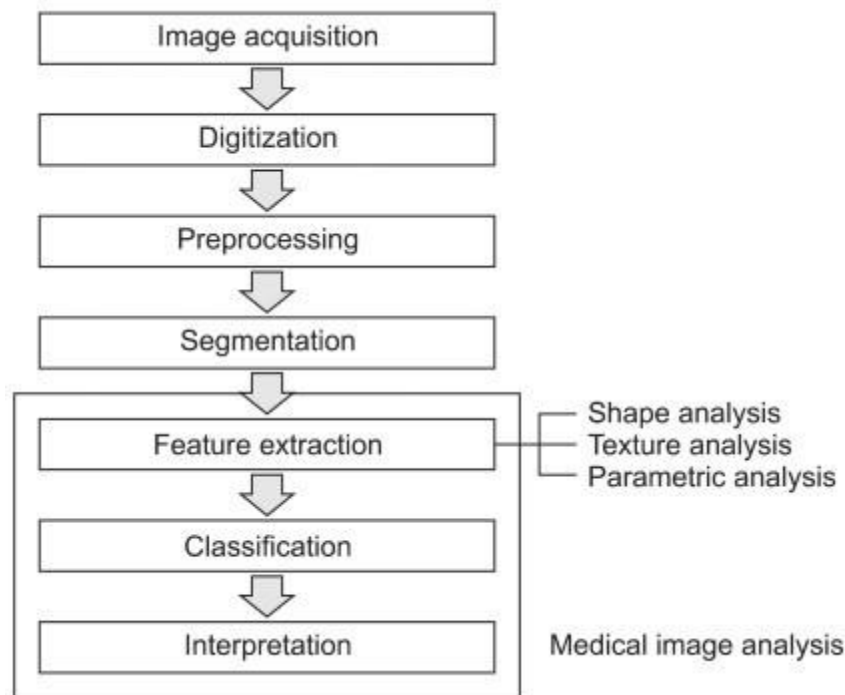
extension, Matplotlib is a graphing library. For usage with embeddingplots in applications, it provides an object-oriented API for general-purpose GUI toolkits like Tkinter, wxPython, Qt, or GTK. Although its use is discouraged, there is another procedural "pylab" interface that is based on a state machine (like OpenGL) and is intended to closely resemble MATLAB. SciPy utilizes Matplotlib.

**SYSTEM ARCHITECTURE**

The system's interactions and the flow of control from one cycle point to another are shown by the architecture structure. There, we will talk about the hardware control flow (Fig. 2) of the system, from image capture to illness diagnosis and display.

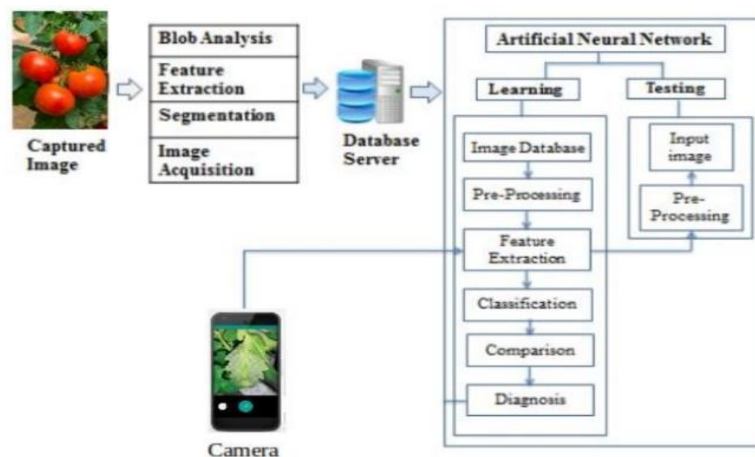
Digital or mobile cameras are used to take pictures of the sick plant leaf. After the picture has been taken, it is uploaded to the system and the image processing step is carried out. This software system receives the final image as an input. This picture serves as the testing data that will be used to assess the precision of our technology. Prior to being put to the test on our machine learning model, the image is first adjusted for size, color, and fit using a variety of methods. Testing on this image is now possible.





**Fig 2:** Steps for Medical Image Analysis

The figure 3 below shows the hardware and software control flow of the system as a whole.



**Fig 3:** Image Capturing through Camera

### Dataset Collection and Creation

The dataset is an enormous collection of raw data that must be trained in order to extract any meaningful information from it. The dataset (Fig. 4) that our system uses consists of a series of photos of different plant leaves that are either healthy or fit into one of the categories for disease:

✓ Tomato Target Spot

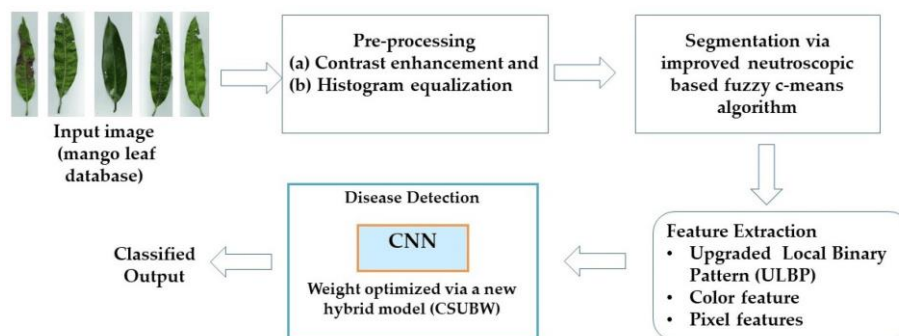
- ✓ Tomato mosaic virus
- ✓ Tomato Bacterial spot
- ✓ Tomato Early blight
- ✓ Tomato healthy
- ✓ Tomato Late blight
- ✓ Tomato Leaf Mold
- ✓ Tomato Septoria leaf spot
- ✓ Tomato Yellow Leaf Curl Virus
- ✓ Tomato Two Spotted Spider Mite

photographs has been scaled, polished, and given a certain quality and dimension. This dataset serves as our training set, governing the platform's initial stages of digital picture processing.



1. Use the pre-composed, gathered, and stored photos in the form of our dataset for training.
2. Test the plant leaf photograph that we took to see whether or not it is infected. The gathered dataset serves as the training set, which is tested against our model of image processing. After then, the model is stored and utilized to test

Page 30



**Fig 5:** Entire mechanism for leaf image processing

## CONCLUSION:

In conclusion, comprehensive methods for the analysis of biomedical images play a pivotal role in advancing medical research, diagnosis, and treatment. By leveraging sophisticated techniques from computer vision, machine learning, and image processing, researchers and healthcare professionals can extract valuable insights from complex biomedical data.

These methods enable the automated detection and characterization of abnormalities, lesions, and anatomical structures in medical images, thereby enhancing the accuracy and efficiency of disease diagnosis and prognosis. Moreover, they facilitate the discovery of biomarkers and patterns indicative of disease progression, treatment response, and patient outcomes.

However challenges remain in the development and application of comprehensive methods for biomedical image analysis. These challenges include the need for large and diverse annotated datasets for training and validation, computational complexity and resource requirements, standardization and reproducibility of analysis pipelines, and ethical considerations regarding patient privacy and data security.

Moving forward, continued research and collaboration across interdisciplinary

domains, including computer science, biology, medicine, and engineering, is essential for addressing these challenges and advancing the state-of-the-art in biomedical imaging analysis. By harnessing the power of technology and innovation, we can unlock new insights into disease mechanisms, improve clinical decision-making, and ultimately enhance patient care and outcomes.

## REFERENCES

1. Sankaran, S.; Mishra, A.; Ehsani, R. "A review of advanced techniques for detecting plant diseases". *Comput.Electron. Agric.* 2010, 72, 1–13. [CrossRef]
2. Chaerani, R.; Voorrips, R.E. "Tomato early blight (*Alternariasolani*): The pathogens, genetics, and breeding for resistance", *J. Gen. Plant Pathol.* 2006, 72, 335–347. [CrossRef]
3. Alvarez, A.M." Integrated approaches for detection of plant pathogenic bacteria and diagnosis of bacterial diseases", *Annu. Rev. Phytopathol.* 2004, 42, 339–366. [CrossRef] [PubMed]
4. Gutierrez-Aguirre, I.; Mehle, N.; Delic, D.; Gruden, K.; Mumford, R.; Ravnkar, M. "Real- time quantitative PCR based sensitive detection and genotype discrimination of Pepino mosaic

- virus”, J. Virol. Methods 2009, 162, 46–55. [CrossRef] [PubMed]
5. Martinelli, F.; Scalenghe, R.; Davino, S.; Panno, S.; Scuderi, G.; Ruisi, P.; Villa, P.; Stropiana, D.; Boschetti, M.; Goudart, L.; et al. “Advanced methods of plant disease detection. A review” ,Agron.Sust. Dev. 2015, 35, 1–25. [CrossRef]
6. Bock, C.H.; Poole, G.H.; Parker, P.E.; Gottwald, T.R. “Plant Disease Sensitivity Estimated Visually, by Digital Photography and Image Analysis, and by Hyperspectral Imaging”. Crit. Rev. Plant Sci. 2007, 26, 59–107.
7. Down To Earth's print edition dated April 1-15, 2019
8. Lefeuvre, Pierre, Darren P. Martin, Gordon Harkins, Philippe Lemey, Alistair JA Gray, Sandra Meredith, Francisco Lakay et al. "The spread of tomato yellow leaf curl virus from the Middle East to the world." PLoS pathogens 6, no. 10 (2010).
9. India Agriculture GDP report from Ministry of Statistics and Programme Implementation (MOSPI)
10. L. S. Puspha Annabel, T. Annapoorani and P. Deepalakshmi, “Machine Learning for Plant Leaf Disease Detection and Classification – A Review,” 2019 International Conference on Communication and Signal Processing (ICCSP), Chennai, India, 2019, pp. 0538-0542.
11. J. D. Pujari, R. Yakkundimath and A. S. Byadgi, “Identification and classification of fungal disease affected on agriculture/horticulture crops using image processing techniques,” 2014 IEEE International Conference on Computational Intelligence and Computing Research, Coimbatore, 2014, pp. 1-4.
12. R. Hari Shankar, A. K. Veeraraghavan, Uvais, K. Sivaraman and S. S. Ramachandran, "Application of UAV for Pest, Weeds and Disease Detection using Open Computer Vision," 2018
13. International Conference on Smart Systems and Inventive Technology (ICSSIT), Tirunelveli, India, 2018, pp. 287-292.
14. Kumar, Meeta, et al. “Survey on techniques for plant leaf classification.” International Journal of Modern Engineering Research 1.2 (2011): 538-544.
15. Fuentes, Alvaro & Lee, Yujeong& Hong, Youngki& Yoon, Sook& Park, Dong. (2016). Characteristics of Tomato Plant Diseases - A study for tomato plant disease identification. Babatunde, Oluleye H., et al. "A genetic algorithm-based feature selection." (2014).
16. F. Y. A. Rahman, S. R. M. S. Baki, A. I. M. Yassin, N. M. Tahir and W. I. W. Ishak, "Monitoring of Watermelon Ripeness Based on Fuzzy Logic," 2009 WRI World Congress on Computer Science and Information Engineering, Los Angeles, CA, 2009, pp. 67-70.
17. P. Pawar, V. Turkar and P. Patil, "Cucumber disease detection using artificial neural network," 2016 International Conference on Inventive Computation Technologies (ICICT), Coimbatore, 2016, pp. 1-5.
18. Zhang, Harry. "The optimality of naive Bayes." AA 1.2 (2004): 3.
19. Z. N. Reza, F. Nuzhat, N. A. Mahsa and M. H. Ali, "Detecting jute plant disease using image processing and machine learning," 2016 3rd International Conference on Electrical Engineering and Information Communication

- Technology (ICEEICT), Dhaka, 2016, pp. 1-6.
20. S. Ramesh et al., "Plant Disease Detection Using Machine Learning," 2018 International Conference on Design Innovations for 3Cs Compute Communicate Control (ICDI3C), Bangalore, 2018, pp. 41-45.
21. Meenakshi K, Safa.M, Krthick T, Sivaranjani N." A novel study of machine learning algorithms for classifying health care data", Research Journal of Pharmacy and Technolgy,
22. Research Journal of Phamacy and Technology, 2017
23. Meenakshi K, Maragatham, G, Agarwal, N., Ghosh, I, " A Data mining Technique for analysing and predicting the success of movie", Journal of Physics: Conference Series, Vol 1000, Issue 1, 2018
24. Saranya, G., & Pravin, A. (2020). A comprehensive study on disease risk predictions in machine learning. International Journal of Electrical and Computer Engineering (IJECE), 10(4), 4217.
25. Saranya, G., Geetha, G., & Safa, M. (2017). E-antenatal assistance care using decision tree analytics and cluster analytics based supervised machine learning. 2017 International Conference on IoT and Application (ICIOT)
26. G Geetha, M Safa, C Fancy, D Saranya "A hybrid approach using collaborative filtering and content based filtering for recommender system", Journal of Physics: Conference Series Vol 1000, Issue 1, 2018.
27. M Srivastava, S Pallavi, S Chandra, G Geetha,"Comparison of optimizers implemented in Generative Adversarial Network (GAN)" International Journal of Pure and Applied Mathematics Volume 119 Issue 12,2018